Univerzita Hradec Králové
Fakulta informatiky a managementu
Katedra informatiky a kvantitativních metod

Využití herní knihovny Bevy ve výuce počítačové grafiky

Diplomová práce

Autor: Bc. Matěj Kolář
Studijní obor: Aplikovaná Informatika

Vedoucí práce: Ing. Bruno Ježek Ph.D

Hradec Králové březen 2026

Prohlášení:

Prohlašuji, že jsem tuto diplomovou práci vypracoval samostatně a uvedl jsem všechny použité prameny a literaturu. V případě využití nástrojů umělé inteligence jsem plně deklaroval způsob jejich využití při vypracování této práce.

V Hradci Králové dne 19.3.2026 Matěj Kolář

Poděkování:

Děkuji vedoucímu diplomové práce, Ing. Brunu Ježkovi, Ph.D. za jeho, odborné rady, cenné připomínky a ochotu, kterou mi věnoval při zpracování této práce.

Abstrakt

Cílem práce je představit knihovnu Bevy a zhodnotit její možný přínos a využití ve výuce počítačové grafiky. Veškeré poznatky a výsledky vychází z Bevy ve verzi 0.15. Změnám ve verzích, které vyšli během psaní práce se věnuje kapitola Vývoj Bevy. V práci je zběžně představen jazyk Rust a knihovna Bevy. Výstupem práce je tento dokument a sada zdrojových kódů ukázek. Jednotlivé ukázky jsou v práci popsány, zhodnoceny a porovnány z hlediska náročnosti implementace s ukázkami napsanými též v jazyce Rust za pomocí knihovny Vulkano.

Abstract

Title: Use of Bevy game engine for teaching computer graphics

The aim of this diploma thesis is to introduce the Bevy library and evaluate its possible contribution and use in teaching computer graphics. All findings are based on Bevy version of 0.15. Changes in versions released during the writing of the thesis are briefly discussed in the chapter „Vývoj Bevy “. This thesis briefly introduces the Rust language and the Bevy library. The output of this diploma thesis is this document and a set of example source codes. The examples are described, evaluated, and compared in terms of implementation complexity with examples also written in the Rust programming language using the Vulkano library.

Key words:

Rust, Bevy, Computer graphics, Game engine, Rendering, Shaders, Education

Obsah

1 Úvod ... 1

2 Počítačová grafika ... 2

2.1 Základní zobrazovací pipeline ... 2

3 Jazyk Rust ... 5

3.1 Historie jazyka ... 5

3.2 Vlastnosti jazyka ... 6

3.3 Ekosystém a struktura projektu ... 9

3.4 Integrace s jinými jazyky ... 9

4 Bevy ... 11

4.1 ECS ... 11

4.2 Zdroje (Resource) ... 14

4.3 Asset server ... 14

4.4 Pluginy ... 14

4.5 Renderovací backendy ... 14

4.6 Koncept světů ... 15

4.7 Materiály ... 16

4.8 WGSL shadery ... 17

4.9 Render graph ... 19

4.10 GUI ... 19

5 Vývoj Bevy ... 20

5.1 Bevy 0.16 ... 20

5.2 Bevy 0.17 ... 21

6 Metodika a zpracování ... 23

6.1 Zpracování ukázek ... 23

6.2 Zpracování Bevy ukázek ... 23

6.3 Využití nástrojů umělé inteligence 23
7 Návrh a implementace grafických úloh 24
7.1 Vytvoření okna, příprava projektu 24
7.2 Obyčejný trojúhelník 24
7.3 Uniform buffer 25
7.4 Geometrie 26
7.5 Textury 27
7.6 Kamera 28
7.7 Modely 28
7.8 Vykreslování více objektů 29
7.9 Osvětlení 29
7.10 Texture array 31
7.11 Compute shader 31
7.12 Geometry shader 32
7.13 Multiple shaders 33
7.14 Post processing 33
7.15 Teselace 35
7.16 Deferred shading 35
8 Bevy ukázky 36
8.1 Camera plugin 36
8.2 ECS a Inspektor 37
8.3 Test výkonu backendů 37
9 Shrnutí výsledků 39
10 Závěr 40
11 Zdroje a literatura 41
12 Přílohy 42

Seznam obrázků

Obr. 1 Zobrazovací řetězec 3

Obr. 2 Schéma entit a komponent 12

Obr. 3 Fáze vykreslování v Bevy 16

Obr. 4 Zjednodušený render graph 19

Obr. 5 Těleso vykreslené s využitím Phongova osvětlovacího modelu 30

Obr. 6 Trojúhelník pro render výstupu post-processing shaderu 34

Obr. 7 Výstup edge detect post-processing shaderu 34

Obr. 8 Výsledky porovnání výkonu 38

Seznam ukázek kódu

Kód 1 Proměnné v jazyce Rust 7

Kód 2 Struktury v jazyce Rust 7

Kód 3 Match statement 8

Kód 4 Příklad ECS – komponenta a vytvoření entity 13

Kód 5 Příklad ECS – systém rotace 13

Kód 6 Definice materiálu 16

Kód 7 WGSL – switch statement 17

Kód 8 Příklad WGSL shaderu 18

Kód 9 Základní struktura aplikace v Bevy 24

Kód 10 Průchozí WGSL fragment shader 25

Kód 11 WGSL fragment shader s uniformní proměnnou 25

Kód 12 Setup 3D scény s kamerou, zdrojem světla a krychlí 26

Kód 13 Fragment shader pro aplikaci textury na těleso 27

Kód 14 Použití Obj pluginu 28

Kód 15 Fragment shader – Phongův osvětlovací model 30

Kód 16 Mapování více textur na více objektů 31

Kód 17 Post-processing sepia fragment shader 34

Kód 18 Snippet pluginu pro ovládání kamery 36

1 Úvod

Počítačová grafika je obor zaměřený na matematiku a algoritmy, které jsou využívány pro generování, reprezentaci a zpracování vizuálních dat. Principy počítačové grafiky jsou využívány v široké škále aplikací. Například v počítačových hrách, filmovém průmyslu, medicíně nebo i v obyčejném uživatelském rozhraní každodenních aplikací. Dá se s jistotou říci, že veškerý vizuální obsah v digitální podobě, zobrazený na obrazovce, přišel do kontaktu s jednou z mnoha forem počítačové grafiky. Proto je důležité, aby o ní měl každý, kdo se chce věnovat informačním technologiím, alespoň základní přehled.

Snaha pochopit principy počítačové grafiky může studenta hlavně v počátcích snadno zahltit, protože se setkává s velkým objemem komplexních informací a algoritmů, které nemusí být na první pohled snadno pochopitelné. Proto je vhodné se stále poohlížet po nových nástrojích a knihovnách, které by výuku udělali pro příchozí studenty přívětivější a zajímavější.

Cílem práce je seznámit čtenáře s knihovnou nebo též herním enginem Bevy, prozkoumat její možné využití pro výuku počítačové grafiky a zhodnotit náročnost implementace ukázkových úloh. Ukázkové úlohy implementují a jsou porovnávány s úlohami z bakalářské práce „Využití API Vulkan v kontextu výuky počítačové grafiky“ [3].

2 Počítačová grafika

Počítačová grafika je oblast, která se zabývá tvorbou a zpracováním grafických informací za pomocí počítačů. V dnešní době se jedná o nepostradatelný obor, s jehož výstupy se, jakkoliv zkušený uživatel setkává téměř neustále. Od grafických rozhraní kioskových aplikací, přes počítačové hry až po vizualizace dat nebo vědecké simulace.

V počítačové grafice rozlišujeme dvě hlavní reprezentace obrazových dat, rastrovou a vektorovou. Rastrový obraz je tvořen obrazovými elementy (pixely) uspořádanými do mřížky, zatím co vektorová grafika je reprezentována pomocí křivek a čar, které jsou prostorově spojité. Vektorovou grafiku není možné běžně zobrazit na rastrovém zařízení, a proto musí projít rasterizací.

Rasterizace je proces, při kterém je obraz za pomocí specializovaných algoritmů převeden z vektorového prostoru do rastrové grafiky. Samotný proces může být pro běžný procesor náročný. Proto je často prováděn pomocí specializovaných grafických čipů, které jsou optimalizovány pro paralelní zpracování vektorových dat.

Základním kamenem vektorové grafiky je souřadnicový systém. Objekty se umisťují do prostoru, který tento systém definuje. V závislosti na požadavcích může být prostor dvou nebo tří rozměrný. Pomocí vhodných matic je poté možné vektorová data transformovat tak, aby docházelo k požadované vizualizaci.

Důležitou součástí moderní grafiky je barva. Stejně jako u fotografie nebo televize, první výstupy počítačové grafiky byly jednobarevné a až s postupem času a vývojem nových technologií se přešlo na grafiku barevnou. Na obrazovkách je barva nejčastěji reprezentována v aditivním formátu RGB. Ale existují i další formáty, které se lépe hodí pro jiné druhy výstupu a vizualizace.

2.1 Základní zobrazovací pipeline

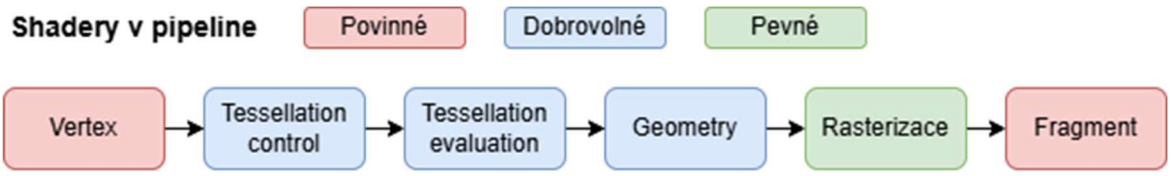
Objekty jsou rozmístěny v rámci vektorového prostoru – scény, s využitím individuálních modelovacích transformačních matic. Zmíněné matice slouží k translaci, škálování, rotaci a jsou aplikovány na homogenizované souřadnice každého vrcholu tělesa. Homogenizace je proces, při kterém jsou 3D souřadnice rozšířeny o další rozměr o hodnotě 1. Výstupem modelovací transformace jsou souřadnice v prostoru scény.

Dalším krokem při renderování scény je převedení souřadnic vrcholů objektů do prostoru kamery, za pomocí dříve specifikované pohledové matice. Pohledová matice zůstává stejná pro všechny vrcholy celé scény v rámci jednoho snímku.

Dále následuje aplikace projekční matice, která způsobuje perspektivní zkreslení. Po této operaci dochází k ořezání částí geometrie ležících mimo zobrazitelný prostor. Pipeline následně provede dehomogenizaci, při níž jsou souřadnice vráceny zpět do 3D prostoru a viewportovou transformaci, která převede vrcholy do souřadnic obrazovky. Výsledné souřadnice vrcholů jsou poté předány k dalšímu případnému zpracování a konečně rasterizaci, čímž vzniká výsledný obraz v pixelech.

Programovatelný zobrazovací řetězec (též rendering pipeline) lze ovlivnit úpravou chování jednotlivých shaderů. Shader je označení jak pro fyzickou výpočetní jednotku, která se nachází v čipu grafické karty, tak název pro program, který se na těchto jednotkách spouští. Shaderové jednotky jsou specializované paralelní ALU procesory, které jsou optimalizované pro hromadné zpracovávání a transformaci dat při renderování scény. Kód shaderu definuje funkci, kterou zobrazovací řetězec pomocí HW jednotky vykonává nad daty v konkrétní fázi zpracování.

Základními typy shaderů jsou vertex a fragment, které tvoří jádro zobrazovacího řetězce. V kombinaci s těmito základními shadery se dále využívají compute, tessellation control, tessellation evaluation, ray tracing nebo mesh shadery. Za zmínku také stojí geometry shader, od kterého se v současné době upouští, a je v novějších verzích grafických knihoven postupně nahrazován mesh shaderem. Na následujícím obrázku lze vidět diagram operací v programovatelném řetězci.



Obr. 1 Zobrazovací řetězec
Zdroj: vlastní zpracování

Moderní grafické karty používají unifikovanou architekturu, kde je HW jednotka shaderu schopna zpracovávat všechny druhy shaderů. Celý řetězec je proto více flexibilní a umožňuje efektivnější využití dostupných zdrojů. U starších zařízení tomu tak nebylo a HW jednotky se dále dělily podle účelu, například na vertex ALU, pixel ALU.

Vertex shader zpracovává každý vrchol vstupních vektorových dat. A typicky při něm dochází k aplikaci modelové, pohledové a projekční matice. Fragment shader se naopak provádí pro každý pixel a používá se například pro texturování objektů.

Tessellační shadery nejsou povinnou součástí pipeline, jejich využití však může vést k snížení výpočetní náročnosti scény. Existující primitivní trojúhelníky modelů jsou rozdělovány na více menších trojúhelníků, v závislosti na nastavení shaderu, čímž dochází k zvýšení rozlišení geometrie. Tato technika je často využívána pro render terénu. Kdy terén v blízkosti kamery má vyšší kvalitu než terén v dáli, u kterého není maximální kvalita žádoucí.

Geometry shader zpracovává celé geometrické primitivy, například trojúhelníky nebo čáry, nikoli jen jednotlivé vrcholy. Může být použit k změnám pozic vrcholů nebo vytváření nových primitiv, čímž z jednoduchého vstupu vznikají složitější geometrické struktury. Typické využití je například pro deformaci geometrie nebo generování vizuálních efektů. Rozšířením a implementací těchto funkcí je možné modifikovat základní chování pro implementaci pokročilejších technik.

3 Jazyk Rust

Jazyk Rust je funkcionální staticky typovaný jazyk, vyvíjený jako výkonná, paměťově bezpečná náhrada za jazyk C. V současné době je již část linuxového jádra přepsána do Rustu, což má pozitivní vliv na šíření jazyka.

Kompilace jazyka je prováděna skrze command line utilitu Cargo. Cargo je oficiální nástroj pro správu balíčků a build projektů v Rustu. Při kompilaci vychází Cargo z konfigurace uložené v soboru cargo.toml, součástí konfigurace jsou i verze externích balíků, které mají být využity a parametry pro build.

3.1 Historie jazyka

Verze 1.0 jazyka Rust byla vypuštěna v květnu roku 2015. Vývoj jazyka započal v roce 2006, kdy do roku 2009 jazyk vyvíjel Graydon Hoare jakožto neveřejný projekt ve svém volném čase. V roce 2009 začal být projekt sponzorován Mozillou a začátkem roku 2012 byla vydána první veřejná verze 0.1. Ve stejném období byl také kompilátor přepsán z jazyka OCaml do jazyka Rust, čímž se jazyk stal takzvaně self-hosted. V čase mezi roky 2012 a 2015 byl jazyk postupně upravován, aby mohla vzniknout první stabilní verze. Do té doby se jazyk příliš často měnil na to, aby mohl být použit pro reálnou aplikaci.

V prvních pěti letech od vydání verze 1.0 začal být jazyk používán i pro jiné komerční projekty mimo Mozillu. Mezi první uživatele se zařadily firmy jako Samsung, Dropbox, Facebook nebo Tilde inc. V roce 2020 došlo během pandemie k restrukturalizaci vývojového týmu, což způsobilo obavy o budoucnost jazyka. V reakci na tyto obavy vznikla, díky podpoře od společností Mozilla, AWS, Google, Huawei a Microsoft, nezisková organizace Rust Foundation, která převzala potřebná práva a zaštítila další údržbu a budoucí vývoj jazyka. Od založení Rust Foundation se počet podporovatelů rozrostl o další společnosti, jako například Sentry, JetBrains nebo ARM.

3.2 Vlastnosti jazyka

3.2.1 Bezpečná správa paměti pomocí vlastnictví

Jednou z hlavních výhod jazyka je bezpečná správa paměti bez potřeby garbage collectoru. Rust toho dosahuje pomocí systému vlastnictví (ownership), který zajišťuje bezpečnou správu paměti v čase kompilace. Když dojde k přesunu hodnoty mezi proměnnými, tak původní proměnná ztrácí vlastnictví a nelze ji nadále využívat. Rust také zavádí koncept půjčky, kdy lze k datům dočasně přistupovat, aniž by došlo ke změně vlastníka.

3.2.2 Systém půjček a dob života

Díky principům ownership, borrow a lifetime jazyk Rust efektivně eliminuje problémy spojené s přístupem k paměti, jako jsou dangling pointers nebo use after free chyby.

Dangling pointer je chyba při které pointer stále ukazuje na prostor paměti, který byl již dříve uvolněn. Pokud se program pokusí prostřednictvím takového pointeru přistupovat k paměti, tak už se jedná o use after free chybu, která může způsobit poškození dat v paměti, neočekávané chování nebo může být zneužita útočníkem pro nebezpečnou manipulaci s pamětí.

Životnost (lifetime) je doba, po kterou zůstává reference (zápůjčka) platná a ukazuje na existující data. K jeho deklaraci používá Rust symbol „ ' “ následovaný libovolným názvem. Kompilátor tak může snadno ověřit, že žádná reference nepřežije proměnnou, na kterou se odkazuje. Ve většině případů je kompilátor Rustu schopen životnost určit sám, ale občas je mu potřeba pomocí `'lifetime_name`.

K označení zapůjčených hodnot (referencí) používá Rust znak „&“, který značí zápůjčku pouze pro čtení (imutable). Pokud je potřeba zapůjčit hodnotu i pro zápis (mutable) použije se zápis „&mut“. Pro rozbalení reference Rust používá symbol „*“.

3.2.3 Statické typy

Rust je staticky typovaný jazyk, tedy všechny proměnné mají předem určený typ v době kompilace. Rust umí typy odvozovat v závislosti na přiřazené hodnotě nebo kontextu, ale u větších projektů je vhodnější typ definovat. Samozřejmostí jsou i generické typy nebo deklarace vlastní datových typů.

3.2.4 Proměnné a datové typy

Deklaraci nové proměnné je potřeba zahájit vhodným klíčovým slovem:

- let – Deklarace běžné proměnné, která je bez slovíčka mut neměnná.
- static – Deklarace globální proměnné.
- const – Deklarace konstanty s fixním typem.
- mut – Proměnná je označena jako tzv. mutable a lze její obsah měnit.

Integery jsou deklarovány jako i32, i64, u32, u64, usize, isize. Písmena u/i napovídají, zda se jedná o unsigned/signed integer a dle následující číslovky lze určit jeho velikost. Rust nativně podporuje 8 až 128bitové integery. Velikost size varianty závisí na velikosti pointeru platformy, pro kterou je kód kompilován. Na 64bitovém systému bude tedy size 64bitů, na 32bitovém 32 bitů, nejméně bude mít však 16 bitů. Datové typy float, double jsou značeny jako f32, f64 a chovají se v souladu s IEEE 754-2008.

Mezi další primitivní datové typy patří 32bitový char, 8bitový bool, nebo statický str. Na řetězec typu str pohlíží standardní knihovna jako na list u8 se statickou délkou a předpokládá kódování UTF-8. Pro řetězce s dynamickou velikostí je lepší využít datový typ String, který je možné modifikovat libovolným způsobem.

Příklad deklarace a definice proměnných:

```
let mut counter: usize = 0;
const LIMIT: u32 = 50;
let path: &str = "/path/dir/file";
let mut name: String = "Jack".to_owned();
```

Kód 1 Proměnné v jazyce Rust
Zdroj: Vlastní zpracování

Rust stejně jako další systémové jazyky podporuje vytváření aliasu datových typů pomocí klíčového slova „type“. Mezi další struktury patří také struct a enum.

```
struct MyStruct {
    number: i32,
    index: usize,
}
enum MyEnum {
    ALICE,
    BOB,
}
type NamesEnum = MyEnum;
type Str = str;
```

Kód 2 Struktury v jazyce Rust
Zdroj: Vlastní zpracování

3.2.5 Řízení toku programu

Základní struktury jako if, else, for, while, loop fungují identicky jako v ostatních jazycích. Velmi užitečnou strukturu, kterou Rust přidává je match. Match je klasický switch statement obohacený o schopnost pracovat nad komplexními strukturami a příjemnější syntaxí.

```
let number:i32 = 5;
match number {
    1 => println!("Jedna"),
    2 => println!("Dva"),
    3 | 4 => println!("Tři nebo čtyři"),
    _ => println!("Jiné číslo"), //default
};
```

Kód 3 Match statement
Zdroj: Vlastní zpracování

3.2.6 Funkce a moduly

Funkce jsou v Rustu definovány pomocí klíčového slova „fn“. Poté následuje název funkce, její parametry a případě typ návratové hodnoty. Rust nemá podporu pro defaultní návratové hodnoty jako například None z Pythonu, pro případy, kdy je none potřeba lze využít přetížení Option<T> jako typ návratové hodnoty.

```
fn soucet(a: i32, b:i32) -> i32{
    a + b
}
```

Na ukázce si lze povšimnout absence klíčového slova return. Rust považuje výstupy bez středníku na konci řádku za návratové hodnoty. Alternativní zápis pomocí returnu by vypadal takto:

```
return a + b;
```

Moduly jsou Rustová obdoba knihoven nebo balíčků u jiných jazyků. Pro import se používá klíčové slovo „use“. Moduly mohou být deklarovány v rámci jednoho souboru pomocí klíčového slova „mod“ nebo pomocí souboru mod.rs v kořenovém adresáři modulu.

3.2.7 Makra

Makra jsou konstrukce umožňující generaci kódu během kompilace. Rust rozlišuje dva druhy maker, deklarativní a procedurální. Název každého deklarativního makra je zakončen symbolem ‚!‘ a používá se obdobně jako běžné funkce. Příkladem je třeba makro println!(), které vypisuje text do konzole pomocí stdout bufferu.

Procedurální makra jsou pokročilejší a transformují kód na úrovni AST. Časté využití je ve spojení s enumy a structy kde jsou makra používána pro generaci pomocného kódu. Například při implementaci debugu, který usnadní převod obsahu na string:

```
#[derrive(Debug)]
struct MyStruct{
    number: i32,
}
```

3.3 Ekosystém a struktura projektu

Rust má moderní ekosystém, který usnadňuje vývoj a správu projektů. Klíčovou součástí je cli utilita Cargo, jedná se o správce balíčků a build systém, který se stará o kompilaci, závislosti a testování. Další důležitou částí ekosystému jsou veřejné úložiště knihoven, jako například Crates.io. Pro správu verzí Rustu se používá nástroj rustup, který umožňuje snadno přepínat mezi různými verzemi.

Standardní Rust projekt má jasně definovanou strukturu. Zdrojové soubory se nacházejí ve složce src/, kde je buď main.rs pro binární aplikace, nebo lib.rs pro knihovny. V kořenovém adresáři projektu se nachází konfigurační soubor cargo.toml. Souboru obsahuje metadata projektu a seznam závislostí a argumenty pro build.

Nový projekt lze pomocí utility Cargo vytvořit příkazem `cargo new my_project`, přičemž Cargo automaticky vygeneruje potřebné soubory a složky. Kompilace projektu se provádí příkazem `cargo build`, zatímco `cargo run` nejen sestaví, ale i spustí aplikaci. Parametr `--release` slouží k buildu optimalizované verze programu.

Testování je v Rustu přímo integrované do jazyka a Cargo poskytuje příkaz cargo test pro automatické spouštění testů. Testy se definují pomocí anotace #[test], která označuje funkce, jež ověřují správné chování kódu. Rust má také silný ekosystém nástrojů pro zlepšení kvality kódu. Clippy slouží jako linter a pomáhá odhalit neoptimální konstrukce, zatímco Rustfmt se stará o automatické formátování kódu podle standardních konvencí.

3.4 Integrace s jinými jazyky

Knihovny napsané v jazyce Rust je možné zkompilovat, tak aby šli využívat i z jiných programovacích jazyků. A naopak, v některých případech je také možné z Rustu volat kód napsaný v jiných jazycích.

K funkcím v Rust knihovně je například možné přistupovat z C/C++, Pythonu nebo Javy. Ale možnosti jsou téměř neomezené. Pokud jazyk podporuje práci s C nebo C++ knihovnami, nebo může být zkompilován tak aby byl přístupný z C/C++, pak je možné ho propojit s Rustem. Na Crates.io lze nalézt knihovny, které tuto integraci usnadňují. Příkladem jsou pyo3, csbindgen, j4r nebo wasm-bindgen.

4 Bevy

Jazyk Rust nespadá do rodiny objektově orientovaných jazyků a Bevy tuto vlastnost odráží ve svém přístupu, který je značně odlišný. Vývojáři označují Bevy engine na svém webu za „data-driven“ pomocí vlastního Bevy ESC systému, jehož předností má být jednoduchost, rychlost a efektivita hlavně díky nativní paralelizaci, která není závislá na zámcích. Pro renderování využívá Bevy knihovnu wgpu.

4.1 ECS

Základními kameny ECS systému jsou, jak již název napovídá, entity, komponenty a systémy. Do jisté míry se dá na prvky ECS dívat jako na instance tříd v kontextu tradičního OOP, avšak tento pohled je milný a může způsobovat problémy. Prvky a jejich vzájemné vztahy jsou rozebrány do většího detailu v následujících kapitolách.

4.1.1 Entity

Entita je nedílná součást ECS systému, na které je závislé fungovaní všech ostatních částí. Entitu si lze představit jako jádro s unikátním ID.

4.1.2 Komponenty

Za komponentu je považován jakýkoliv struct, označený jako komponenta. Komponenty slouží ke skladování informací, nebo identifikaci entit pomocí vlastností.

4.1.3 Systémy

Systémy jsou funkce, které jsou volány podle předem určených pravidel a dle uvážení plánovače ECS systému. Pořadí běhu těchto funkcí tak není nijak zaručené a funkce se snaží běžet tak paralelně, jak jen je to možné. Výjimkou k tomuto pravidlu jsou funkce, které mají pomocí zmíněných pravidel běhu určené závislosti na jiných systémech.

Různá pravidla běhu a zaváděcí funkce můžou být využita k změně chování a optimalizaci celé aplikace. Například setup systémy není nutné a ani žádoucí volat opakovaně. Systémy pro čtení vstupů od uživatele je nutné provádět co nejčastěji pro snížení vstupní latence. A u fyzikální systémů a agentových systémů je žádoucí, aby jejich běh byl spíše plynulejší i za cenu snížené frekvence, nežli častý a nepředvídatelný. Stejně jako snímková frekvence.

4.1.4 Integrace entit, komponent a systémů

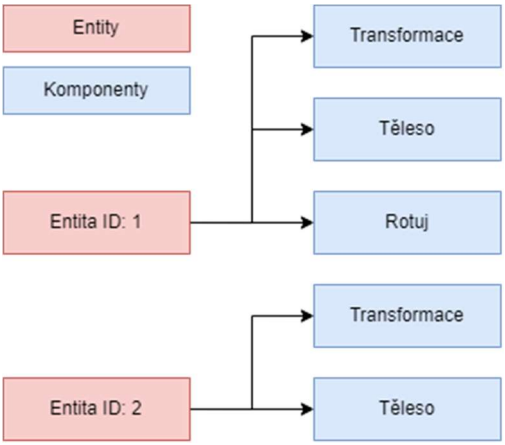
Pokud je entita jádro, pak komponenty jsou vrstvy navázané na toto jádro. Každá komponenta nějakým způsobem ovlivňuje výsledné chování entity a slouží i jako identifikátor pro systémy, které mají potřebu s touto entitou nakládat.

Příkladem může být jedna z často používaných komponent – Transform. Komponenta určuje pozici a rotaci entity v prostoru. Při renderu je použita k vytvoření transformační matice, která se aplikuje na všechny další tělesa, které jsou navázané na stejnou entitu.

Příklad integrace

V této části bude implementován jednoduchý systém, který má za úkol každý tik pootočit zvolené objekty o 0.1 rad kolem osy Y. Zvolené objekty musí mít komponentu Rotuj. Ostatní jsou ignorovány.

V následující sekci práce je znázorněno schéma provázání entit a komponent tak, aby došlo k rotaci entity ID:1, zatím co entita ID:2 bude statická. Dále se zde nachází implementace stejné struktury pomocí kódu.



Obr. 2 Schéma entit a komponent
Zdroj: vlastní zpracování

```
// Komponenta rotace
#[derive(Component)]
struct Rotuj;

// Setup funkce, která řeší spawn entit a přiřazení komponent
fn setup(mut commands: Commands) {
    // Rotující entita
    commands.spawn((
        Transform::default(),
        GlobalTransform::default(),
        Rotuj, // Vlastní komponenta
    ));

    // Statická entita
    commands.spawn((
        Transform::default(),
        GlobalTransform::default()
    ));
}
```

Kód 4 Příklad ECS – komponenta a vytvoření entity
Zdroj: Vlastní zpracování

Po vytvoření entit je potřeba ještě implementovat samotný systém pro provádění rotace. Systém si pomocí dotazu zažádá o transformace všech entit, které mají komponentu rotuj a poté aplikuje rotaci.

```
fn rotuj(mut query: Query<&mut Transform, With<Rotuj>>){
    for mut transform in query.iter_mut(){
        transform.rotate_local_y(0.1);
    }
}
```

Kód 5 Příklad ECS – systém rotace
Zdroj: Vlastní zpracování

Setup systém a rotační systém je potřeba do Bevy aplikace zavést tak, aby se prováděli ve vhodné fázi programu. K určení okamžiku spuštění slouží již definované zaváděcí funkce, které systémy registrují do konkrétních fází scheduleru. Pro účely ukázkové implementace je potřeba do aplikace zavést systémy setup a rotuj. Systém setup je potřeba provádět pouze jednou při startu aplikace, zatím co systém rotuj, se musí provádět opakovaně s každým tikem. Pro usnadnění implementace bude snímková frekvence a frekvence tiků plně provázaná, s každým snímkem proběhne jeden tik.

Využijeme tedy funkce `add_startup_system`, pro zavedení setup systému a funkce `add_systems`, s parametrem Update pro zavedení rotace. Parametr Update informuje scheduler, že systém je potřeba volat s každým snímkem.

```
app.add_startup_system(setup);
app.add_systems(Update, rotuj);
```

4.2 Zdroje (Resource)

Resource je z pohledu Bevy struct, který je globální a unikátní v rámci světa a slouží ke skladování informací používaných při běhu. Může se jednat o konfigurace, assety, proměnné a další. Informace uložené ve zdrojích se mohou a nemusí za běhu měnit. Registrace zdroje probíhá skrze metodu `app.insert_resource()`. Mutable i imutable přístup ke zdrojům probíhá skrze systémy. Avšak vlastníkem zdrojů je stále Bevy.

4.3 Asset server

Pro načítání souborů Bevy využívá asset server. Jedná se zdroj, který zajistí včasné načtení souboru a zabrání jejich případné duplikaci. Funkce load vrací univerzální asset handle, pomocí kterého se reference na soubor předává v rámci ECS. Díky asset handles nevznikají potíže s vlastnictvím, protože handle je pouze odkaz, pomocí kterého v kombinaci s asset serverem lze přistupovat k obsahu souboru.

4.4 Pluginy

Pro zvýšení přehlednosti a modularity používá Bevy systém pluginů. Pluginy jsou do aplikace přidány při startu a obsahují předem připravené systémy, do kterých vývojář nemusí zasahovat, jejich případná konfigurace probíhá pomocí zdrojů.

Pluginy lze získat z různých zdrojů. Bevy obsahuje předpřipravené pluginy a skupiny pluginů, které zajišťují základní funkcionality enginu. Jedná se například o skupiny DefaultPlugins a MinimalPlugins nebo pluginy jako InputPlugin, DevToolsPlugin, LogPlugin a další. Vývojáři si mohou vytvářet vlastní pluginy nebo využívat existující pluginy z knihoven, které rozšiřují Bevy o pokročilejší funkce.

4.5 Renderovací backendy

Bevy pro renderování používá WebGPU API zpřístupněné skrze knihovnu wgpu, a proto nativně podporuje několik renderovacích backendů, které je možné při startu aplikace snadno měnit.

Primární backendy, které mají plnou podporu jsou: Vulkan, Metal a DirectX 12. V případech, kdy aplikace cílí na prohlížeč pomocí WebAssembly je k dispozici WebGPU backend. Sekundární renderovací backend, který sice nemá plnou podporu, ale lze jej i tak využívat je OpenGL.

WebGPU a wgpu

Wgpu je knihovna pro jazyk Rust, která slouží jako rozhraní mezi aplikací a WebGPU API, které zajišťuje komunikaci s konkrétním renderovacím backendem.

WebGPU je navrženo s důrazem na přenosnost mezi operačními systémy a různým hardwarem, a proto neimplementuje všechny funkcionality, které by jinak jednotlivé renderovací backendy poskytovali. Mezi nedostupné funkce patří například geometry nebo tessellační shadery.

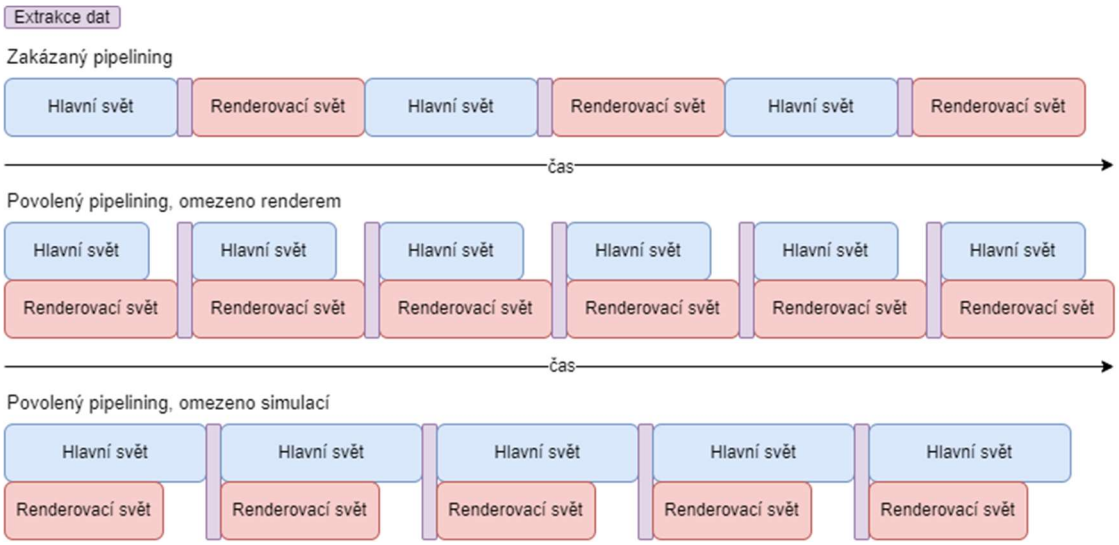
Podporované WebGPU shadery – vertex, fragment a compute jsou psány v jazyce WGSL (WebGPU Shading Language) [8], kód shaderu je poté při kompilaci překládán do formátu, který je kompatibilní se zvoleným renderovacím backendem.

4.6 Koncept světů

Svět je z pohledu Bevy prostor, který reprezentuje stav aplikace, obsahuje všechny zdroje, entity a další informace. Základní Bevy aplikace má světy dva. Jeden hlavní, ve kterém probíhá simulace a jeden vykreslovací, který zodpovídá za renderování. Většina běžných operací probíhá nad světem simulačním, až na operace, které pracují přímo s renderem, ty je potřeba provádět ve světě renderovacím. Příkladem takové operace je například pipeline pro post processing.

Přesun dat mezi světy probíhá v závislosti na nastavení pipeliningu. Pokud je pipelining povolen, tak se na začátku cyklu data ze simulačního světa extrahují do světa renderovacího. Oba světy poté paralelně provádějí vlastní agendu. Před započnutím dalšího cyklu musí rychlejší z dvojice světů čekat na ten pomalejší, aby mohlo dojít k bezpečné synchronizaci všech dat. Na následujícím obrázku jsou zachyceny obě varianty čekání před synchronizací.

Při zakázaném pipeliningu nejprve proběhne cykl ve světě simulačním, poté dojde k přesunu dat a probíhá cykl ve světě vykreslovacím. Celá smyčka tak probíhá sériově. Tento přístup snižuje počet renderovaných snímků mezi akcí uživatele a viditelnou změnou na obrazovce za cenu nižší frekvence aktualizace, což může být v některých typech aplikací žádoucí.



Obr. 3 Fáze vykreslování v Bevy
Zdroj: vlastní zpracování

4.7 Materiály

Materiály v Bevy slouží k určování vzhledu zobrazovaných objektů. Toho je dosaženo pomocí dvojice vertex a fragment shaderů, které jsou materiálem specifikované. Materiály také definují proměnné, které ovlivňují chování přidružených shaderů. Typickým příkladem těchto proměnných jsou barvy, textury nebo jiné uniform hodnoty. Bevy obsahuje několik předpřipravených materiálů pro různé účely. Nebo je možné definovat vlastní materiály s vlastními shader funkcemi.

Všechny materiály musí v nějaké podobě implementovat trait AsBindGroup, aby mohly být nastavené hodnoty proměnných správně předány do paměti grafické karty.

```
#[derive(Asset, TypePath, AsBindGroup, Debug, Clone)]
struct CustomMaterial {
    #[uniform(0)]
    example: i32,
}
impl Material2d for CustomMaterial {
    fn fragment_shader() -> ShaderRef {
        SHADER_FILE.into()
    }
}
```

Kód 6 Definice materiálu
Zdroj: Vlastní zpracování

4.8 WGSL shadery

Bevy umožňuje využití WebGPU Shading Language, zkráceně WGSL pro implementaci vlastních shaderových funkcí. Kód napsaný v jazyce WGSL je překládán do vhodného formátu v závislosti na zvoleném renderovacím backendu. Jazyk se syntaxí velmi podobá Rustu, bez systému zápůjčky, hlavní rozdíly budou v této kapitole více rozebrány.

Entry points

Entry points neboli vstupní body jsou anotace, které označují funkce, jenž mají být použity pro spuštění shaderu. Anotace začínají znakem @ a mohou mít libovolný název za předpokladu, že odpovídá vstupnímu bodu, který je v pipeline definován. Příkladem vstupních bodů, které používá Bevy jsou @vertex nebo @fragment.

Binding points

Binding points je označení pro anotace proměnných, které jsou využívány pro přístup k datům ve WGSL, jako jsou například buffery, textury nebo samplery. Pomocí anotací @group(n) a @binding(m) se určuje, ve které skupině a na které pozici se data nachází.

```
@group(0) @binding(1) var<uniform> time_modifier: f32;
```

Obdobou je anotace @location(n), která slouží k řízení toku dat mezi shadery a používá se na vstupech a výstupech entry point funkcí.

Řízení toku – switch

WGSL používá klasický switch statement, na rozdíl od Rustu, který používá match, ostatní struktury pro řízení toku se chovají identicky.

```
switch a {
    case 1: {
        return 1;
    }
    case 2, 3: {
        return 6;
    }
    default: {
        return 5;
    }
}
```

Kód 7 WGSL – switch statement
Zdroj: Vlastní zpracování

Proměnné

WGSL používá tři základní druhy proměnných, a to: const, let a var. Const slouží k definici hodnot známých už v době kompilace a nelze je měnit. Proměnné deklarované pomocí let jsou neměnné během běhu, jejich hodnota se nastaví při vytvoření a dále už ji nelze upravovat. Var představuje klasickou proměnnou s možností měnit její hodnotu.

Příklad shaderu

Následující WGSL kód aplikuje pomocí vertex shaderu posun +-5 jednotek podél osy Y na každý vertex v závislosti na uniformní proměnné time, která je s každým snímkem aktualizována a obsahuje čas který uběhl od spuštění aplikace. Fragment shader též využívá uniform time aby moduloval barvu pixelu násobením číslem 0 až 1.

```
//import pomocných funkcí z Bevy
#import bevy_sprite::mesh2d_functions::{get_world_from_local,
mesh2d_position_local_to_clip};

@group(2) @binding(0) var<uniform> time: f32; //uniform času
struct VertexOutput {
    @builtin(position) position: vec4<f32>,
    @location(4) color: vec4<f32>,
};
struct Vertex { //Data vertexu - poskytuje Bevy
    @builtin(instance_index) instance_index: u32,
    @location(0) position: vec3<f32>,
    @location(4) color: vec4<f32>,
};

@vertex //Vertex shader entrypoint
fn vertex(in: Vertex) -> VertexOutput {
    let offset_y: f32 = sin(time)*5.0; // posun v rozsahu -+5

    // původní pozice + posun Y podle času
    let new_pos = vec3<f32>(in.position.x,
        in.position.y + offset_y, -1.0);

    var out: VertexOutput;
    //MPV trasformace
    out.position = mesh2d_position_local_to_clip(
        get_world_from_local(in.instance_index),
        vec4<f32>(new_pos, 1.0)
    );
    out.color = in.color; //Předání barvy dál
    return out;
}

@fragment //Fragment shader entrypoint
fn fragment(in: VertexOutput) -> @location(0) vec4<f32> {
    return in.color * (0.5 + 0.5 * sin(time)); //modulace barvy
}
```

Kód 8 Příklad WGSL shaderu
Zdroj: Vlastní zpracování

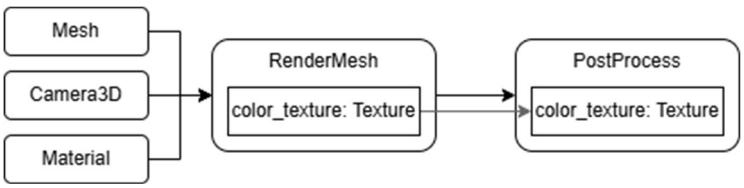
4.9 Render graph

Renderovací graf je modulární způsob, jakým Bevy řeší skládání a volání příkazů pro grafickou kartu. Jednotlivé uzly grafu si mezi sebou předávají potřebné zdroje a provádí nad nimi předem stanovené operace. Struktura render grafu také podporuje tvorbu pod-grafů, které mohou být zabaleny a prováděny z jednoho uzlu.

Po stránce implementace je renderovací graf bez-stavová struktura, která udává, jakým stylem jsou jednotlivé renderovací uzly propojeny. Při renderu scény iterátor prochází tuto strukturu a provádí operace definované jednotlivými uzly. Render graph je acyklický graf složený ze tří prvků – Node, Edge a Slot. Uzly (Node) jsou zodpovědné za samotné renderovací instrukce a jejich volání, hrany (Edge) udávají, jakým způsobem jsou mezi sebou uzly propojeny a sloty jsou používány k definici zdrojů, které renderovací uzly využívají nebo vytvářejí.

Pro ilustraci lze uvést zjednodušený příklad. Graf obsahuje uzel RenderMesh, který vykresluje objekty scény, a uzel PostProcess, který aplikuje na obsah color_texture slotu vizuální efekty. Hrany mezi uzly určují pořadí zpracování, nejprve se provede RenderMesh, poté PostProcess. Slot color_texture slouží jako buffer, do kterého RenderMesh zapíše svůj výstup, aby ho mohl PostProcess zpracovat.

Kvalitní dokumentace k této základní funkcionalitě Bevy bohužel chybí a existující články jsou zastaralé a nepoužitelné pro novější verze knihovny.



Obr. 4 Zjednodušený render graph
Zdroj: Vlastní zpracování

4.10 GUI

Bevy v současné verzi obsahuje zárodky vlastního uživatelského rozhraní, které však zatím nejsou v takovém stavu, aby se daly snadno a pohodlně používat. Naproti tomu plugin Bevy-Egui, postavený na knihovně egui, již nabízí uživatelsky přívětivé a plně funkční grafické rozhraní napsané v jazyce Rust. Je dobře integrovaný do Bevy a vyžaduje minimální konfiguraci ze strany vývojáře pro dosažení funkčního grafického rozhraní.

5 Vývoj Bevy

Bevy je aktivní projekt, a neustále se vyvíjí. V následujících kapitolách jsou popsány změny a aktualizace, o které byla knihovna Bevy rozšířena během psaní této práce a jejich případný vliv na výsledek práce. Úplné seznamy změn, v originálním znění, je možné nalézt na domovské stránce Bevy [9], v sekci news.

5.1 Bevy 0.16

Verze 0.16 přinesla několik nových funkcionalit. Žádné z nich nemají znatelný dopad na závěr práce. Změny se týkají optimalizace výkonu a QoL. Za zmínku stojí hlavně následující výběr z velkého množství změn.

GPU driven rendering

Bevy 0.16 automaticky používá GPU driven rendering, u komplexních scén tak dochází k výraznému snížení vytížení CPU. GPU driven rendering přesouvá část výpočetních úloh, které se dříve prováděly na procesoru, přímo na grafickou kartu. Díky tomu lze efektivněji zpracovávat velké množství objektů ve scénách. Implementace je pro uživatele transparentní – není potřeba žádná konfigurace nebo zásah do existujícího kódu.

Occlusion culling

Experimentální implementace occlusion culling není ještě plně odladěná a má problémy s přesností. Jejím cílem je zamezit vykreslování objektů, které nejsou aktuálně viditelné, protože jsou zakryty jinými objekty ve scéně. Tímto způsobem může engine dosáhnout vyšší efektivity při renderování a šetřit výpočetní výkon. Vzhledem k experimentální povaze je však nutné počítat s možnými vizuálními chybami a nestabilním chováním, které mohou být v budoucích verzích opraveny.

ECS vztahy

Aktualizace přinesla možnost lépe provázat více entit mezi sebou. Staré řešení vyžadovalo specializované struktury a nebylo snadno použitelné. Momentálně je nový systém omezený na 1:N, avšak v budoucnu je plánováno ho rozšířit na M:N. Typickým příkladem je auto jakožto hlavní entita a jeho kola jako podřazené entity.

Vylepšení virtuální geometrie

Systém virtuální geometrie Meshlet inspirovaný Unreal Engine 5 Nanite zaznamenal ve verzi 0.16 zlepšení výkonu. Hlavně díky použití nových algoritmům a rozšíření GPU API. Nová verze implementuje efektivnější zpracování trojúhelníků a lepší organizaci dat, což snižuje nároky na GPU. Zlepšení se týká i práce s viditelností objektů a odstraňování neviditelných fragmentů.

Snazší profilování GPU.

Diagnostické informace o renderu jsou nyní dostupné v nástroji Tracy pro snazší optimalizaci výkonu. Tato změna umožňuje sledovat časování úloh prováděných na grafické kartě. Vývojář tak získává přehled o průběhu vykreslování, i když je zatím podpora omezena a specializované nástroje jsou stále účinnější.

Klonování entit

Klonování entit bylo již dříve možné, ale bylo pomalé a vyžadovalo registraci všech komponent. Nyní má Bevy nativní podporu pro efektivní klonování pomocí makra `#[derive(Clone)]` a volání funkce. EntityClonerBuilder může být využit ke konfiguraci, jakým způsobem bude klonování probíhat, například pomocí filtrování komponent nebo omezením rekurze u entit které využívají nový systém vztahů.

5.2 Bevy 0.17

Verze Bevy číslo 0.17 vyšla 30 září 2025 a přinesla řadu novinek, které nemají zásadní dopad na závěry této práce. Podobně jako v předchozí verzi se změny zaměřují rozšíření renderovacích možností, optimalizaci výkonu a vylepšení vývojářského komfortu.

Bevy Solari

S verzí 0.17 byla do enginu zavedena experimentální podpora pro hardwarově akcelerovaný raytracing. Crate bevy_solari ještě není připravena pro produkční nasazení, ale její vznik značí první kroky Bevy k využití raytracingu namísto tradičních osvětlovacích modelů.

DLSS

Uživatelé, kteří vlastní grafickou kartu série RTX značky nVidia nyní mohou využívat pokročilý DLSS anti-aliasing založený na strojovém učení. Funkcionalita je dostupná pouze pro renderovací backend Vulkan na operačních systémech Windows a Linux. Obdobné technologie, které nabízí ostatní výrobci HW jako AMD, Intel, nebo Apple. Nebyly implementovány, avšak dle autora je možné, že budou implementovány v budoucích verzích.

Viewport node

Další novinkou je dedikovaný viewport node, který umožňuje snazší renderování výstupu kamery přímo do textury. Tím se výrazně zjednoduší práce s více pohledy a kamerami, například při tvorbě zrcadel, obrazovek nebo náhledových oken v editorech.

6 Metodika a zpracování

Cílem této práce je představit knihovnu Bevy a zhodnotit její možné využití pro výuku počítačové grafiky na základě dostupných informací a praktických zkušeností nabytých během implementace ukázkových úloh.

6.1 Zpracování ukázek

Ukázky jsou založené na ukázkách z práce „Využití API Vulkan v kontextu výuky počítačové grafiky“ [3] a implementovány pomocí Bevy. Cílem re-implementace ukázek je získání podkladů pro pozdější zhodnocení náročnosti implementace a přívětivosti enginu. Rendering engine Bevy některé operace velmi zjednodušuje, ztěžuje a v některých případech i znemožňuje. Proto bylo upuštěno od původního záměru implementovat ukázky co nejvěrněji i po stránce kódu. A při implementaci byl kladen důraz hlavně na implementaci stejného tématu a poté na podobnost výstupu.

Některé pokročilé techniky vyžadují modifikaci nebo nahrazení exitujícího render grafu a pipeline. Vytváření vlastní pipeline je z větší části práce s knihovnou WGPU, která je zapouzdřena pomocí Bevy. A společně s render grafem jsou tyto funkce nedostatečně zdokumentovány, aby bylo možné je rozumně využít. Proto byly implementovány pouze ty úlohy, které již existující pipeline a render graf modifikují a rozšiřují. Úlohy vyžadující celkovou re-implementaci vlastního render grafu a pipeline nejsou řešeny.

6.2 Zpracování Bevy ukázek

Bevy ukázky slouží k představení některých specifických funkcionalit knihovny Bevy. Téma jednotlivých ukázek je voleno dle vlastního uvážení. Jedná se o rozšiřující obsah, který není možné zařadit do převzatých Vulkano ukázek, a zároveň není vhodné ho z práce vynechat.

6.3 Využití nástrojů umělé inteligence

Při psaní práce byly využity modely Claude Haiku 3.5 a GPT-4o za účelem zlepšení srozumitelnosti práce a pro kontrolu gramatických chyb. AI nebylo využito při psaní kódu ukázek. Příklady použitých promptů:

- Je možné následující text napsat lépe? <text>
- Obsahuje následující text pravopisné chyby? <text>

7 Návrh a implementace grafických úloh

Kapitola se zabývá ukázkami, jejich implementací v Bevy, vysvětlením a porovnáním s ukázkami představenými v práci „Využití API Vulkan v kontextu výuky počítačové grafiky“ [3]. Ukázky demonstrují stejnou funkcionalitu, jakou implementují referenční příklady uvedené ve výše zmíněné bakalářské práci. Proto jsou si některé části kódu podobné, zatím co jiné se můžou výrazně lišit. Samotné ukázky by měli sloužit jako stavební bloky, pomocí kterých je možné začít vytvářet vlastní aplikaci.

7.1 Vytvoření okna, příprava projektu

Velmi krátká ukázka p00 obsahuje základní struktury potřebné pro vytvoření okna, s předem určenými parametry. Vlastnosti okna jsou definovány pomocí structu Window, ve kterém lze nastavit rozlišení okna, titulek, režim okna, vsync nebo vlastnosti kurzoru. Struct je dále předán WindowPluginu a aplikace je spuštěna. Z této základní struktury budou vycházet všechny následující ukázky.

```
fn main() {
    // Nastavení okna
    let window_settings = Window {
        resolution: WindowResolution::new(800.0, 600.0),
        title: "P00 Window example".to_string(),
        mode: WindowMode::Windowed,
        present_mode: PresentMode::AutoVsync,
        ..default()
    };
    // Vytvoření aplikace
    let mut app = App::new();
    app.add_plugins(DefaultPlugins.set(WindowPlugin {
        primary_window: Some(window_settings),
        ..default()
    }));
    // Start
    app.run();
}
```

Kód 9 Základní struktura aplikace v Bevy
Zdroj: Vlastní zpracování, ukázka p00

7.2 Obyčejný trojúhelník

Ukázka p01 implementuje rendering dvourozměrného trojúhelníku. Ukázka je oproti ukázce p00 rozšířena o nastavení renderovacího backendu pomocí RenderPluginu, definici vlastního materiálu pro podporu upraveného fragment shaderu a setup systém, který obstará jednorázové nastavení scény. Fragment shader slouží pouze jako minimální průchodová funkce, jež předává dál barvu poskytnutou

předpřipraveným Bevy vertex shaderem. Následující ukázka WGSL kódu obsahuje Fragment shader implementovaný aplikací.

```
struct FragmentInput {
    @location(4) frag_color: vec4<f32>
};
@fragment
fn fragment(in: FragmentInput) -> @location(0) vec4<f32> {
    return in.frag_color;//Předávání barvy dále
}
```

Kód 10 Průchozí WGSL fragment shader
Zdroj: Vlastní zpracování, ukázka p01

Porovnání s Vulkano

Na této ukázce je možné dobře pozorovat rozdíl v náročnosti implementace. Samotná logika ukázky je minimální, a i přesto vyžaduje Vulkano složitější strukturu projektu. Bevy na pozadí samozřejmě obsahuje větší množství kódu který se musí provádět, ale vývojář není nijak zatížen tímto kódem.

7.3 Uniform buffer

Kód ukázky p02 dále rozšiřuje ukázku p01 o uniform buffer, pro nastavení násobiče barvy trojúhelníku. Obsah uniform bufferu je možné měnit za běhu aplikace pomocí uživatelského rozhraní.

UI je v tomto případě řešeno knihovnou Egui, která umožňuje příjemnou tvorbu jednoduchých i pokročilejších uživatelských rozhraní. Rendering pipeline není potřeba nijak upravovat díky integraci knihovny Egui pomocí Bevy pluginu Bevy-Egui. Následující WGSL snippet obsahuje deklaraci uniform proměnné a její využití ve fragment shaderu.

```
// Uniform proměnná
@group(2) @binding(0) var<uniform> color_mul: vec4<f32>;

struct FragmentInput {
    @location(4) frag_color: vec4<f32>
};

@fragment
fn fragment(in: FragmentInput) -> @location(0) vec4<f32> {
    return  in.frag_color*color_mul;
}
```

Kód 11 WGSL fragment shader s uniformní proměnnou
Zdroj: Vlastní zpracování, ukázka p02

Rozdíl oproti Vulkano implementaci

Vulkano ukázka využívá uniform buffer pro MVP matice a časovou složku animací. Bevy umožňuje využít interní vertex shader, který již MVP řeší, a proto se ukázka soustředí pouze na využití bufferu k nastavení násobiče barvy.

7.4 Geometrie

Ukázka p03 je první, která implementuje 3D scénu. Přechod z 2D do 3D scény je snadný, stačí pouze změnit druh kamery z Camera2d na Camera3d. Scéna obsahuje statickou krychli, která je přidána pomocí kombinace struktur Mesh3d, Cuboid, MeshMaterial3d a Color. Mesh3d ve spojení se strukturou Cuboid slouží k definici tvaru tělesa pomocí vertex a index bufferu, zatím co MeshMaterial3d a Color určují jeho vzhled. Materiály poskytované knihovnou Bevy obsahují podporu pro integrovaný osvětlovací systém. Tento osvětlovací systém je v ukázce využit a do scény je přidán i zdroj světla PointLight.

Na rozdíl od předchozích ukázek není využíván explicitní vertex a index buffer pro definici tvaru tělesa ale struct Cuboid. Bevy obsahuje předpřipravené definice základních těles pro usnadnění práce s knihovnou.

Porovnání s Vulkano

Ukázka se liší v tělesech, které jsou ve scéně použity, způsobem, jakým jsou vrcholy těles definovány a využitím osvětlovacího modelu.

```
   // Tvorba kamery na pozici [20:20:20]
let mut tr = Transform::from_xyz(20., 20., 20.);
// Kamera směřuje pohledem na souřadnice [0:0:0]
tr = tr.looking_at(Vec3::ZERO, Vec3::Y);
commands.spawn((Camera3d::default(), tr));

// Vytvoření zdroje světla na pozici [8:16:8]
commands.spawn((
    PointLight {
        shadows_enabled: true,
        ..default()
    },
    Transform::from_xyz(8.0, 16.0, 8.0),
));

// Vytvoření krychle o délce hrany 5 na pozici [0:0:0]
commands.spawn((
    Mesh3d(meshes.add(Cuboid::new(5., 5., 5.))),
    // Barva krychle je bílá
    MeshMaterial3d(materials.add(Color::srgb(1.0, 1.0, 1.0))),
    Transform::from_xyz(0., 0., 0.),
));
```

Kód 12 Setup 3D scény s kamerou, zdrojem světla a krychlí
Zdroj: Vlastní zpracování, ukázka p03

7.5 Textury

Výstupem ukázky p04 je render koule, na kterou je aplikována textura Země. Textura byla získána z Wikipedia – Blue Marble 2002[4]. Samotná ukázka není náročná na implementaci, většina kódu je takzvaný boilerplate – kód který je nutný pro běh, ale obsahově se liší minimálně. Aplikace textury na těleso je v ukázce řešena pomocí parametru base_color_texture materiálu StandardMaterial. V ukázce p04b je dosaženo stejného výstupu za pomocí vlastního materiálu a fragment shaderu. Následující snippet obsahuje kód fragment shaderu zodpovědný za aplikaci textury na těleso.

```
//WGSL fragment shader
#import bevy_pbr::forward_io::VertexOutput

@group(2) @binding(0)
var color_texture: texture_2d<f32>;
@group(2) @binding(1)
var color_sampler: sampler;

@fragment
fn fragment(in: VertexOutput) -> @location(0) vec4<f32> {
    return textureSample(color_texture, color_sampler, in.uv);
}

//Bevy material
#[derive(Asset, TypePath, AsBindGroup, Clone)]
pub struct TexturedMaterial {
    #[texture(0)]
    #[sampler(1)]
    pub texture: Handle<Image>,
}

impl Material for TexturedMaterial {
    fn fragment_shader() -> ShaderRef {
        "shader.wgsl".into()
    }
}
```

Kód 13 Fragment shader pro aplikaci textury na těleso
Zdroj: Vlastní zpracování, ukázka p04b

Porovnání s Vulkano

Rozdíl spočívá především v tom, jakým způsobem je řešeno mapování textury na 3D objekt. V Bevy ukázce stačí načíst texturu pomocí asset serveru a přiřadit ji k materiálu, který následně zajistí aplikaci textury na mesh pomocí vlastního nebo předpřipraveného fragment shaderu. V případě knihovny Vulkano je potřeba manipulovat s bufferem pro texturu a řešit implementaci pipeline a shaderu které budou s texturou pracovat.

7.6 Kamera

Implementovat vlastní kameru není v ukázce p05 nutné. Bevy nabízí konfigurovatelné komponenty pro 3D a 2D kamery, které jsou plně integrované do ECS a ve většině případů vyžadují minimum úprav. Rotace a pohyb kamery je řešen transformací entity, se kterou je komponenta kamery svázána. V ukázce je aplikace transformace řešena ve funkci camera_controller_system, která je do aplikace zavedena jako systém, volaný s každým renderovacím cyklem. Funkce samotná si pomocí Query získá entitu kamery, přečte ze zdrojů a eventů informace o vstupech a aplikuje odpovídající transformaci na entitu kamery, čímž dojde ke změně pohledu v dalším renderovacím cyklu. Na této ukázce je založena ukázka b01 z kapitoly 8.1, která implementuje vlastní univerzální plugin pro pohyb kamery.

Porovnání s Vulkano

Největší rozdíl mezi ukázkami spočívá v tom, jakým způsobem je řešen pohyb kamery a aktualizace pohledové matice. V Bevy provedeme transformaci entity kamery za pomocí systému a engine automaticky zajistí vše potřebné. Ve Vulkano ukázce je nutné ručně implementovat matice pohledu a projekce, aktualizovat uniform buffery a zajistit jejich předání do vertex shaderu, kde dochází k aplikaci matic na jednotlivé vrcholy.

7.7 Modely

Bevy nativně podporuje načítání gltf scén, ale pro načtení jiných formátů objektů je potřeba načítací funkci implementovat, nebo stejně jako v případě ukázky p06, využít plugin. Bevy obj je plugin pro načítání .obj souborů, je možné ho snadno získat za pomocí systému crates a cargo. Plugin stačí do aplikace zavést při jejím startu, a poté lze .obj modely načítat pomocí asset serveru, bez potřeby další konfigurace nebo manipulace s daty. Jeho použití je tedy velmi snadné.

```
// import
use bevy_obj::ObjPlugin;

// fn main – setup pluginu
app.add_plugins((ObjPlugin, camera_plugin::CameraControlPlugin));

// fn setup – načtení modelu
let mesh = Mesh3d(asset_server.load("teapot.obj"));
```

Kód 14 Použití Obj pluginu
Zdroj: Vlastní zpracování, ukázka p06

Porovnání s Vulkano

Při implementaci ukázky pomocí Bevy stačí pouze využít existující plugin pro načtení OBJ modelu. Zato Vulkano, musí implementovat kompletní parsování OBJ souboru a správu bufferů pro vrcholy a indexy. Celý proces je tak pracnější a vyžaduje více práce s datovými strukturami a grafickým API.

7.8 Vykreslování více objektů

Ukázku p07 není nutné implementovat, protože Bevy již problematiku vykreslování více objektů řeší nativně v renderovacím systému. Jednotlivé techniky je možné implementovat pomocí zásahu do render grafu a vytvořením vlastních pipeline. Nejedná se však o běžné použití Bevy. Ukázka by se musela více soustředit na knihovnu WGPU než na Bevy, což není cílem této práce.

Tělesa jsou v Bevy běžně vykreslována pomocí loop renderingu. Pokud však existuje více entit, které sdílejí mesh a materiál, pak jsou tyto entity automaticky vykreslovány pomocí techniky instancování.

Offset rendering není v Bevy nijak implementován. V případě potřeby by bylo nutné, jak již bylo zmíněno, vytvořit vlastní render pipeline, který by plně nahradil Bevy pipeline.

7.9 Osvětlení

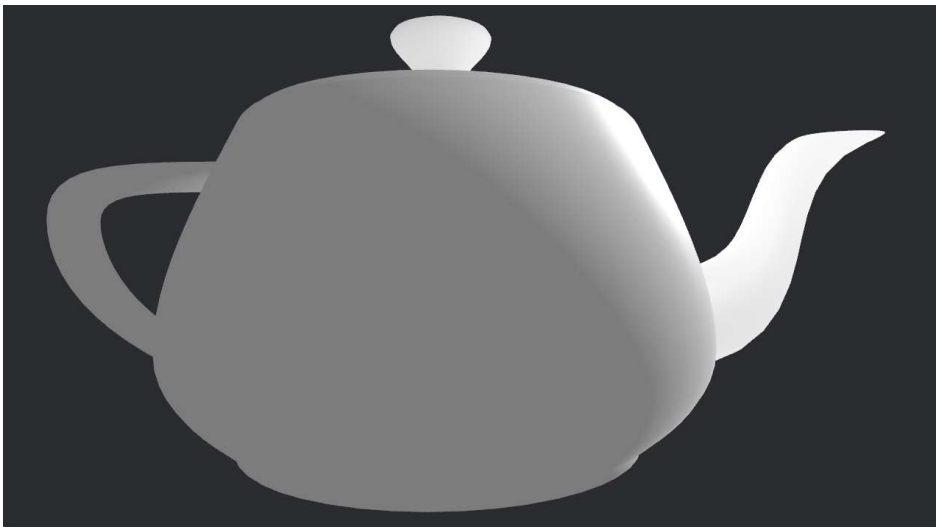
Na ukázce p08 je dobře vidět dříve zmíněné zjednodušení vývoje, které Bevy poskytuje. Všechny předchozí ukázky, které aplikují na tělesa nativní Bevy materiály, již mají implementované i osvětlení, které funguje díky dříve zmíněným nativním materiálům a přidáním entity s komponentou DirectionalLight, jenž je zdrojem světla ve scéně. Bevy využívá osvětlovací model založený na PBR v kombinaci s Cook-Torrance a GGX pro výpočet odlesků.

Pokud by bylo v této ukázce použito nativní osvětlení poskytované enginem Bevy, tak by ukázka ztrácela smysl. Z toho důvodu je v p08 implementován Phongův osvětlovací model řešený pomocí vlastního fragment shaderu.

Systém pro ovládání kamery se nachází ve vlastním pluginu. A proto je součástí ukázky i systém camera_update_system, který se stará o aktualizaci uniformní proměnné camera_pos. Fragment shader načítá pozici kamery z tohoto bufferu a využívá ji pro výpočet osvětlení pomocí Phongova modelu. Následující ukázka kódu obsahuje implementovaný fragment shader v jazyce WGSL.

```
@group(2) @binding(0) var<uniform> light_pos: vec3<f32>;
@group(2) @binding(1) var<uniform> camera_pos: vec3<f32>;
@group(2) @binding(2) var<uniform> ambient_color: vec3<f32>;
@group(2) @binding(3) var<uniform> light_color: vec3<f32>;
@group(2) @binding(4) var<uniform> specular_value: f32;

struct FragmentInput {
    @builtin(position) position: vec4<f32>,
    @location(0) normal: vec3<f32>,
    @location(1) world_position: vec3<f32>,
};
@fragment
fn fragment(in: FragmentInput) -> @location(0) vec4<f32> {
    let normal = normalize(in.normal);
    let light_dir = normalize(light_pos - in.world_position);
    let view_dir = normalize(camera_pos - in.world_position);
    let reflect_dir = reflect(-light_dir, normal);

    // Phong
    let ambient = ambient_color;
    let diffuse = max(dot(normal, light_dir), 0.0) * light_color;
    let specular =
        pow(
            max(dot(view_dir, reflect_dir), 0.0),
            specular_value
        ) * light_color;

    let final_color = ambient + diffuse + specular;
    return vec4<f32>(final_color, 1.0);
}
```

Kód 15 Fragment shader – Phongův osvětlovací model
Zdroj: Vlastní zpracování, ukázka p08



Obr. 5 Těleso vykreslené s využitím Phongova osvětlovacího modelu
Zdroj: Vlastní zpracování, ukázka p08

Porovnání s Vulkano

Ve Vulkanu je nutné implementovat nejen vlastní shadery, ale také všechny buffery a bindingy pro předávání pozic kamery a světla. Implementace Phongova modelu v Bevy je výrazně jednodušší díky abstrakci enginem.

7.10 Texture array

Ukázku, která využívá texture array nebylo nutné implementovat, protože systém materiálů v Bevy nevyžaduje žádné speciální úpravy, aby bylo možné mapovat více textur na více objektů. Stačí přiřadit handle materiálu k dalšímu objektu. Nebo případně vytvořit novou instanci materiálu, pokud se má textura lišit.

```
// Načtení textury A
let texture_handle_a = asset_server.load("a.png");
// Načtení textury B
let texture_handle_b = asset_server.load("b.jpg");

// Vytvoření materiálu
let material_a = materials.add(StandardMaterial {
    base_color_texture: Some(texture_handle_a),
    ..Default::default()
});
let material_b = materials.add(StandardMaterial {
    base_color_texture: Some(texture_handle_b),
    reflectance: 0.5,
    perceptual_roughness: 0.5,
    ..Default::default()
});

// Aplikace textur pomocí materiálu
commands.spawn((
    Mesh3d(<nějaký mesh A>),
    MeshMaterial3d(material_a),
));
// jiný mesh s stejnou texturou jako předchozí
commands.spawn((
    Mesh3d(<nějaký mesh B>),
    MeshMaterial3d(material_a),
));

// stejný mesh s jinou texturou
commands.spawn((
    Mesh3d(<nějaký mesh B>),
    MeshMaterial3d(material_b),
));
```

Kód 16 Mapování více textur na více objektů
Zdroj: Vlastní zpracování

7.11 Compute shader

Compute shader není nativně v Bevy podporován, a proto je potřeba implementovat vlastní pipeline a render graph. V těchto úlohách slouží Bevy pouze jako prostředník zapouzdřující interní WGPU funkce.

Compute shader A

Ukázka p10a implementuje jednoduchý compute shader, který má za úkol získat minimum, maximum a součet hodnot ze vstupního pole. Výsledek je poté uložen do výstupního bufferu, a jeho obsah vypsán do konzole. Bevy ECS systém v tomto případě

značně komplikuje implementaci. Díky zapouzdření Wgpu není možné nad bufferem operovat přímo. Výstupní data se po doběhnutí compute shaderu nachází v paměti GPU a nastává potřeba je přesunout do běžné RAM, kde budou opět přístupná pro CPU. Bevy za tímto účelem implementuje readback, který však probíhá asynchronně a data mohou být i několik renderovacích cyklů pozadu. Toho si lze všimnout i ve výstupu ukázky, kdy readback několik cyklů vrací původní hodnoty, které byly do output bufferu vloženy při inicializaci, než dojde k získání výsledku a ukončení programu.

```
Expected: sum:-74 min:-10 max:9
Readback result: [0, 0, 0]
... opakování předchozího řádku 5x
Readback result: [0, 0, 0]
Readback result: [-74, -10, 9]
Result: sum:-74 min:-10 max:9
```

Compute shader B

V ukázce p10b je implementována jednoduchá simulace částic založená na compute shaderu z referenční ukázky. Bevy není na podobný typ aplikací uzpůsobené, a proto je potřeba si nadefinovat vlastní compute a render pipeline a render nodes, které se budou spouštět při běhu programu.

Tato ukázka byla jedna z nejnáročnějších na implementaci. Tvorba vlastních render pipelines a modifikace render grafu není dobře zdokumentována. Velké množství ukázek na internetu využívá starších verzí Bevy, které danou problematiku řeší jinak než současná verze. Při testování ukázky nad jiným HW a OS, než na kterém byla vyvíjena, se navíc ukázalo, že ukázka nevykresluje výstup simulace.

Porovnání s Vulkano

Vulkano nabízí více možností a detailnější management jednotlivých částí pipeline. Proto je vhodnější pro aplikace se zaměřením na compute shadery. Bevy sice minimalizuje množství potřebného kódu, ale značně tím omezuje možnosti a komplikuje implementaci.

7.12 Geometry shader

Tuto úlohu nebylo možné v Bevy implementovat, protože knihovna WGPU, kterou Bevy používá pro renderování nepodporuje geometry shadery.

7.13 Multiple shaders

Úloha p12 renderuje tři identické trojúhelníky pomocí tří odlišných dvojic vertex/fragment shaderu pro dosažení odlišných výstupů s identickými vstupy.

Samotná implementace této ukázky byla snadná a pouze navazuje na implementaci vlastního shaderu za pomocí materiálu, která byla představena v předcházejících úlohách. Každá dvojice vertex a fragment shaderu je přiřazena k jednomu vlastnímu materiálu, který je definovaný v čase kompilace. Protože shadery materiálu není možné měnit nebo nastavovat při běhu aplikace.

Porovnání s Vulkano

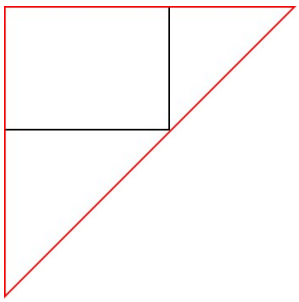
Díky existujícímu systému materiálů, který přináší Bevy, je možné snadno využívat více shaderů během jednoho vykreslovacího cyklu. Stačí pouze definovat více materiálů a přiřadit je jednotlivým meshům. Při použití knihovny Vulkano je nutné pro každý shader vytvořit a spravovat pipeline, zajistit načtení a kompilaci shaderů a explicitně řídit, který pipeline a které shadery se použijí při vykreslování konkrétního objektu.

7.14 Post processing

Bevy nativně implementuje několik různých post processing funkcí, ukázka p13 se proto soustředí na implementaci vlastních funkcí než na využití existujících.

V ukázce je post processing implementován pomocí renderování do textury. Všechny implementované efekty se aplikují na celou obrazovku bez rozdílu, proto je možné pro výstup použít jediný trojúhelník, který pokrývá celou plochu obrazovky. Do části tohoto trojúhelníku se vykresluje obraz z kamery, zpracovaný jedním ze tří post processing fragment shaderů. Vertex shader pro trojúhelník, který obstará i výpočet správných UV souřadnic obrazovky je již v Bevy k dispozici a není ho proto třeba implementovat.

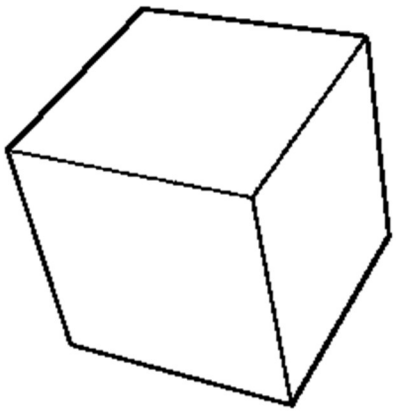
Na obrázku lze vidět červeně trojúhelník, a černě prostor obrazovky, do kterého se výstup vykresluje, dále následuje ukázka WGSL shaderu pro sepia filtr.



Obr. 6 Trojúhelník pro render výstupu post-processing shaderu
Zdroj: Vlastní zpracování

```
#import bevy_core_pipeline::fullscreen_vertex_shader::Fullscreen
VertexOutput

@group(0) @binding(0) var screen_texture: texture_2d<f32>;
@group(0) @binding(1) var texture_sampler: sampler;

@fragment
fn fragment(in: FullscreenVertexOutput) -> @location(0) vec4<f32> {
    var color: vec4<f32> = textureSample(
                                    screen_texture,
                                    texture_sampler,
                                    in.uv);

    var gray: f32 = dot(color.rgb, vec3<f32>(0.3, 0.59, 0.11));
    var sepia: vec3<f32> = vec3<f32>(gray) * vec3<f32>(1.2, 1.0, 0.8);

    sepia.r = clamp(sepia.r, 0.0, 1.0);
    sepia.g = clamp(sepia.g, 0.0, 1.0);
    sepia.b = clamp(sepia.b, 0.0, 1.0);
    return vec4<f32>(sepia, color.a);
}
```

Kód 17 Post-processing sepia fragment shader
Zdroj: Vlastní zpracování, ukázka p13



Obr. 7 Výstup edge detect post-processing shaderu
Zdroj: Vlastní zpracování, ukázka p13

Porovnání s Vulkano

V Bevy je post-processing řešen pomocí přidání vlastního uzlu do render grafu a jednoduchým přepínáním pipeline, která se má na výstup aplikovat. Ve Vulkano ukázce je nutné vykreslit scénu do offscreen framebufferu a výstupní texturu následně použít v dalším průchodu pro post-processing. To znamená více explicitního kódu a detailnější kontrolu všech kroků. Obě ukázky implementují v nějaké podobě vlastního správce post-processing efektu, který se má aplikovat.

7.15 Teselace

Pro teselační shadery nemá WGPU, a tudíž ani Bevy podporu. Teoreticky je možné teselační shadery zkompilovat do SPIR-V formátu a poté je vložit přímo do renderovací pipeline. Ale toto řešení je považováno za vysoce experimentální, omezené pouze na Vulkan backend a oficiálně není podporované. Level of detail objektů bude v Bevy řešen pomocí virtuální geometrie, která je stále ve vývoji.

7.16 Deferred shading

Bevy nativně implementuje pouze forward rendering. Technika deferred shading tedy není v základu podporována. Pro její implementaci by bylo nutné nahradit celý předpřipravený render graf a pipeline vlastním řešením.

8 Bevy ukázky

Kapitola se zabývá popisem ukázek, které implementují a představují prvky vázané na samotnou knihovnu Bevy.

8.1 Camera plugin

Ukázka b01 obsahuje minimální implementaci vlastního pluginu pro pohyb kamery tak, aby stačilo pouze zavést plugin do aplikace a označit kameru, která se má pohybovat. Obsahem knihovny je marker komponenta, která neobsahuje žádná data, a slouží pouze k označení kontrolované kamery. Samotný plugin pouze přidává jeden systém, který vychází ze systému pro pohyb kamery původně implementovaného v ukázce p05.

```
// Komponenta pro označení ovládané kamery
#[derive(Component)]
pub struct ControlledCamera;

// Plugin pro zavedení systému kamery do aplikace
pub struct CameraControlPlugin;

impl Plugin for CameraControlPlugin {
    fn build(&self, app: &mut App) {
        app.add_systems(Update, camera_controller_system);
    }
}

// Zaváděný systém zodpovědný za aktualizaci polohy kamery
fn camera_controller_system(
    mut query: Query<&mut Transform,
        (With<Camera3d>, With<ControlledCamera>)>,
    time: Res<Time>,
    keyboard: Res<ButtonInput<KeyCode>>,
    mut mouse: EventReader<MouseMotion>,
    primary_window: Query<&Window, With<PrimaryWindow>>,
) {
   ...logika pro pohyb kamery
}
```

Kód 18 Snippet pluginu pro ovládání kamery
Zdroj: Vlastní zpracování, ukázka b01

8.2 ECS a Inspektor

V ukázce b02 je využita externí knihovna Bevy inspector egui, která slouží k prohlížení a manipulaci s Bevy světem skrze UI. Ve světě se krom defaultních zdrojů a entit nachází 2D kamera, skrze kterou se na scénu díváme a několik sprite entit, které jsou reprezentovány barevnými čtverci. Každá entita reprezentována čtvercem je pojmenována Sprite <id> a obsahuje komponentu RotateComponent. Entity s komponentou RotateComponent jsou rotačním systémem rotovány rychlostí, která je v komponentě nastavena. Tuto rychlost lze, stejně jako další parametry, upravit skrze UI inspektoru. Cílem této ukázky je poukázat na existenci inspektoru, který je v návodech často opomíjen a může výrazně usnadnit vývoj a debug aplikace. Pro jeho aktivaci stačí přidat crate do projektu a zavést plugin do aplikace.

```
use bevy_inspector_egui::quick::WorldInspectorPlugin;
...
app.add_plugins(WorldInspectorPlugin::new());
```

U vlastních komponent je nutné přidat makro reflect a případně i derive default, pokud není trait již implementován jinak.

```
#[derive(Component, Reflect, Default)]
#[reflect(Component)]
struct MyComponent;
```

8.3 Test výkonu backendů

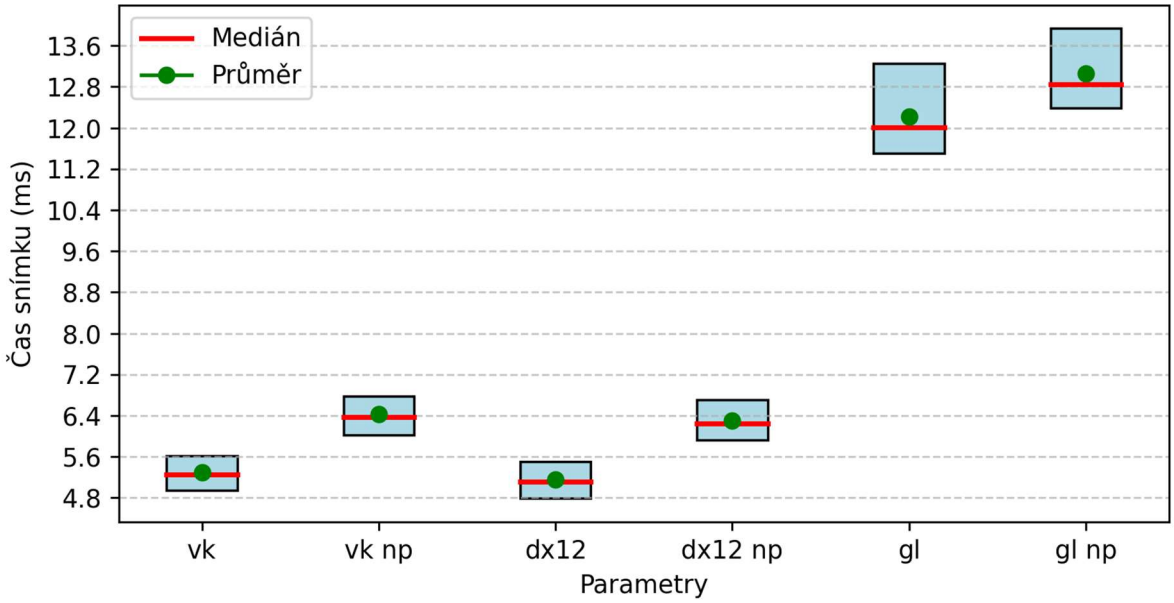
V ukázce b03 je implementována scéna s velkým počtem různě rozmístěných těles. Pomocí vstupních argumentů lze nastavit pipelining a renderovací backend, který bude využit pro test. Během 30 sekund od běhu prvního cyklu dochází k zaznamenání času každého snímku pomocí zdroje Time a funkce delta_secs, výsledky jsou poté vypsány ve formátu [maximum, minimum, medián, 5. percentil, 95. percentil, průměr]. Jednotky výstupních hodnot jsou sekundy.

8.3.1 Výsledky měření výkonu

Graf na obrázku č. 4 znázorňuje naměřené hodnoty pro jednotlivé backendy. Hodnoty jsou převedeny na milisekundy a platí, že čím nižší hodnota, tím rychleji se scéna renderovala. Horní a dolní okraj modrého boxu představují 95. a 5. percentil. Zelené kolečko značí průměr, červená čára medián. Parametr „np“ symbolizuje běh testu bez pipeliningu hlavního a renderovacího světa (popsáno v kapitole 4.6). Backendy jsou značeny zkratkami, vk – Vulkan, dx12 – DirectX12 a gl – OpenGL.

Backend Metal je určený pro operační systém macOS, a proto ho nebylo možné otestovat na stejné konfiguraci počítače.

OpenGL má v testu výhodu, nejedná se totiž o primární renderovací backend s plnou podporou, a proto při testu správně nefunguje osvětlení scény. I s touto výhodou však OpenGL skončilo na posledním místě. Měření byla prováděna na počítači s operačním systémem Windows 10, 16 GB operační pamětí, procesorem AMD Ryzen 7 1700 a grafickou kartou AMD RX 7800 XT.



Obr. 8 Výsledky porovnání výkonu
Zdroj: Vlastní zpracování

9 Shrnutí výsledků

Na základě vlastních zkušeností, získaných při implementaci ukázek v Bevy, lze konstatovat, že většinu ukázek bylo možné implementovat snadněji než jejich ekvivalenty z práce „Využití API Vulkan v kontextu výuky počítačové grafiky“ [3]. Je to zejména proto, že při práci s Vulkanem je nutné vytvářet více vlastních struktur a explicitně realizovat kroky, které Bevy řeší automaticky. To se týká zejména inicializace pipeline, správy paměti a zdrojů, synchronizace mezi CPU a GPU či organizace renderovacích průchodů. Bevy tyto mechanismy zapouzdřuje a zpřístupňuje pomocí renderovacího grafu a architektury ECS.

Ukázky teselace, deffered shadingu, geometry shaderů nebo vykreslování více objektů nebylo možné implementovat ze dvou hlavních důvodů. Prvním je nedostatečná dokumentace funkcí v jádru Bevy enginu, zejména v oblasti správy renderovacího grafu, komunikace s WGPU a renderování scény. Druhým důvodem je absence podpory geometry a tessellation shaderů ze strany WGPU. Pro implementaci těchto ukázek by bylo vhodnější využít low-level knihovnu, která poskytuje přímý, nízko-úrovňový přístup k samotnému renderovacímu backendu.

Díky WGPU je možné přepínat renderovací backendy, ale při testování této funkcionality nad kódy ukázek bylo zjištěno, že aplikace využívající jiný backend, než Vulkan jsou při běhu nestabilní. V některých případech nastali problémy s překladem WGSL shaderů a shader se choval jinak, než by měl. V jiných došlo ke kritickým chybám při komunikaci s GPU, což způsobilo okamžitý pád aplikace. Tyto chyby se zdají být závislé na kombinaci renderovacího backendu, HW počítače a OS. Proto byly všechny ukázky, až na ukázku sloužící pro testování výkonu, řešeny pouze pro renderovací backend Vulkan.

Bevy i přes různé nedostatky plní svůj účel jakožto game engine, který má za úkol z větší části odstínit problematiku renderování a optimalizace. Vývojář se tak může soustředit na vývoj výsledné aplikace jako takové. Proto bych knihovnu doporučil spíše pro výuku předmětu se zaměřením na počítačové hry, nebo jazyk Rust, nežli pro výuku základů počítačové grafiky a renderování.

10 Závěr

Cílem této práce bylo seznámit čtenáře s herním enginem Bevy, představit jeho základní principy a odpovědět na otázku, zda jej lze využít pro výuku počítačové grafiky. Součástí práce bylo taktéž představení programovacího jazyka Rust a vytvoření sady ukázek, které implementují různé funkce Bevy.

Knihovna Bevy není vhodné řešení pro výuku počítačové grafiky ze dvou hlavních důvodů – problém s jazykem Rust a problém se samotným Bevy. Studenti by se museli naučit pracovat s jazykem Rust, který není svojí syntaxí a fungováním kompatibilní s ostatními předměty, které vyučují a využívají převážně klasické objektově orientované jazyky (Java, C# a další…).

Bevy je rozhodně velmi zajímavý projekt, ale engine stále není dostatečně vyspělý pro reálné využití. V době psaní této práce neustále vycházejí aktualizace, které přináší nové optimalizace a funkce, ale zároveň značně mění fungovaní knihovny jako celku. Verze jsou mezi sebou nekompatibilní a migrace mezi verzemi často vyžaduje značné zásahy do projektu. Na tyto časté změny trpí hlavně malé veřejné knihovny a pluginy. Vývojáři, kteří se knihovně věnují ve svém volném čase, často nestíhají kód aktualizovat, a jsou neustále pozadu. Některé knihovny jsou proto pouze udržovány pro starší verzi Bevy. Jiné čekají dlouho na aktualizace. A v krajních případech je i vývoj a veškerá podpora knihovny ukončena.

Bevy game engine zapouzdřuje nízko-úrovňové funkce samotných renderovacích backendů, čímž usnadňuje vývojáři práci, ale student se pak učí spíše pracovat s game enginem jako takovým než principy počítačové grafiky, na kterých engine stojí.

Myšlenka přepínání grafického backendu je rozhodně zajímavá, avšak během psaní a testování 3D ukázek se funkcionalita ukázala být nedoladěná. Aplikace se po změně renderovacího backendu chovali nepředvídatelně a v krajních případech nešli spustit.

11 Zdroje a literatura

[1] Bevy Foundation, Bevy documentation [online]. Dostupné z: https://docs.rs/bevy/0.14.2/bevy/index.html

[2] KLABNIK, Steve, NICHOLS, Carol. The Rust Programming Language [online]. Dostupné z: https://doc.rust-lang.org/stable/book/title-page.html

[3] PROKOP, Dominik. Využití API Vulkan v kontextu výuky počítačové grafiky [online]. Dostupné z: https://theses.cz/id/il01k0/.

[4] Wikipedia, Blue Marble 2002.png [online]. Dostupné z: https://en.wikipedia.org/wiki/File:Blue_Marble_2002.png

[5] gfx-rs, Wgpu documentation [online]. Dostupné z: https://docs.rs/wgpu/latest/wgpu/

[6] Ida Borisova, Unofficial Bevy Cheat Book [online]. Dostupné z: https://bevy-cheatbook.github.io/credits.html

[7] Bevy foundation, Bevy examples [online]. Dostupné z: https://bevyengine.org/examples/

[8] World Wide Web Consortium, WebGPU Shading Language [online]. Dostupné z: https://www.w3.org/TR/2025/CRD-WGSL-20250611/

[9] Bevy Foundation [online]. Dostupné z: https://bevy.org

[10] Rust Foundation [online]. Dostupné z: https://rustfoundation.org/

[11] KLABNIK, Steve. The History of Rust [online]. Dostupné z: https://dl.acm.org/doi/10.1145/2959689.2960081

[12] THOMPSON, Clive. How Rust went from a side project to the world's most-loved programming language [online]. Dostupné z: https://www.technologyreview.com/2023/02/14/1067869/rust-worlds-fastest-growing-programming-language/

12 Přílohy

1) Struktura projektu
2) Návod na spuštění ukázek

Struktura projektu

- doc – Dokument diplomové práce ve formátu pdf a docx
- examples – Zdrojové kódy ukázek
- media – Obrázky a diagramy použité v práci

Všechny soubory jsou dostupné na adrese:

https://github.com/GlummixX/Dimploma_thesis

Návod na spuštění ukázek

Návod předpokládá, že na zařízení je již nainstalovaný jazyk Rust, jeho závislosti a potřebné nástroje. Pokud tomu tak není, navštivte prosím https://www.rust-lang.org/tools/install. Jednotlivé příkazy je nutné spouštět v adresáři examples, a ne v kořenovém adresáři repozitáře.

Pomocí následujícího příkazu je možné a spustit libovolnou ukázku:

```
cargo run --bin <název_složky_ukázky>
```

Například:

```
cargo run --bin p01_basic_triangle
```

Pro maximální výkon je potřeba zkompilovat ukázku v režimu release pomocí parametru -r. Kompilace v režimu release může trvat déle v závislosti na výkonu zařízení.

```
cargo run -r --bin <název_složky_ukázky>
```

Ukázku je také možné pouze zkompilovat pomocí příkazu build. Parametr „-r“ opět slouží pro aktivaci režimu release:

```
cargo build -r --bin <název_složky_ukázky>
```

Výsledný binární soubor bude umístěn do složky examples/target/release. Pokud byl soubor kompilován bez parametru „-r“, bude umístěn do složky examples/target/debug.

„Zadání práce“, které každý student projednal před započetím přípravy závěrečné práce se svým vedoucím práce a vložil do STAGu, je součástí odevzdávané elektronické (popř. tištěné, je-li to požadováno) verze závěrečné práce. Zadání práce student získá z informačního systému eVŠKP (https://ris.uhk.cz/evskp/).